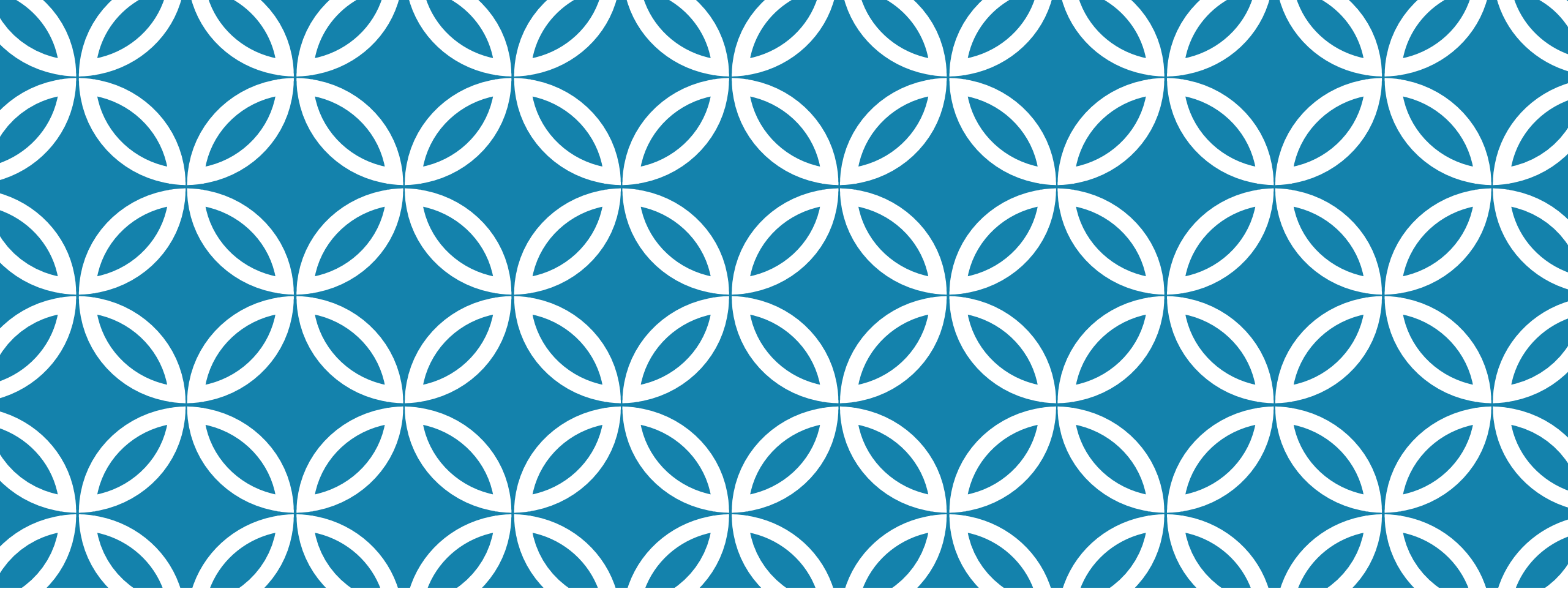




KERAS LIBRARY

dr : heba elhadidy

Eng : Mariyam khashapa

KERAS LIBRARY

What is Keras?

Keras is an open-source, high-level deep learning API written in Python. It runs on top of **TensorFlow** and allows developers to build and train **neural networks** with minimal code.

It is widely used because of its **ease of use, flexibility, and fast prototyping capabilities**.

◆ Basic Example: Building a Simple Neural Network

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Define a simple neural network
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Print model summary
model.summary()
```

◆ What are Layers in Keras?

In **Keras**, a **layer** is a fundamental building block of a neural network.

Layers process input data, apply transformations, and pass the output to the next layer.

Each layer performs a specific function, such as **computing activations**, **extracting features**, or **reducing dimensionality**.

◆ Types of Layers in Keras

Keras provides various types of layers for different tasks:

Dense (Fully Connected) Layer

Used in **feedforward neural networks (MLP)**. Every neuron is connected to all neurons in the previous layer.

Convolutional Layers (Conv2D)

Used in **CNNs (Convolutional Neural Networks)** for **image processing**.

Recurrent Layers (LSTM, GRU)

Used in RNNs (Recurrent Neural Networks) for sequential data (e.g., text, time-series).

summary

Layer

Dense

Conv2D

LSTM / GRU

Embedding

Dropout

Flatten

Purpose

Fully connected layer for general tasks

Extracts image features

Processes sequential data (NLP, time-series)

Converts words to vector representations

Prevents overfitting

Converts multi-dimensional input to 1D



◆ Popular Use Cases

- ◆ **Image Classification** – Using CNNs (e.g., ResNet, VGG, EfficientNet)
- ◆ **Natural Language Processing (NLP)** – Using LSTMs, GRUs, Transformers
- ◆ **Time Series Forecasting** – Predicting stock prices, weather patterns
- ◆ **Reinforcement Learning** – Training AI agents (e.g., AlphaGo)
- ◆ **Anomaly Detection** – Fraud detection, cybersecurity

Digit Classification Using MLP (Multi-Layer Neural Network - MLP)

This example uses the MNIST database to classify hand image libraries (0-9).

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# تحميل بيانات MNIST
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# تطبيع البيانات
x_train, x_test = x_train / 255.0, x_test / 255.0

# بناء النموذج
model = keras.Sequential([
    layers.Flatten(input_shape=(28, 28)), # تحويل الصورة إلى متجه
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])
```

تجميع النموذج

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

تدريب النموذج

```
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

تقييم النموذج

```
test_loss, test_acc = model.evaluate(x_test, y_test)  
print(f"Test accuracy: {test_acc:.4f}")
```

Building a CNN Model for Image Classification (CIFAR-10)

This example uses convolutional neural networks (CNNs) on the CIFAR-10 dataset.

```
# تحميل البيانات
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

# تطبيع البيانات
x_train, x_test = x_train / 255.0, x_test / 255.0

# بناء النموذج
model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```



تجميع النموذج

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

تدريب النموذج

```
model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

تقييم النموذج

```
test_loss, test_acc = model.evaluate(x_test, y_test)  
print(f"Test accuracy: {test_acc:.4f}")
```

LSTM Network for Text Classification (IMDB)

This example uses an LSTM network to process and classify texts (positive and negative reviews).

```
from tensorflow.keras.preprocessing import sequence

# تحميل بيانات IMDB
(x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(num_words=10000)

# توحيد طول التسلسلات النصية
maxlen = 200
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)

# بناء النموذج باستخدام LSTM
model = keras.Sequential([
    layers.Embedding(10000, 128, input_length=maxlen),
    layers.LSTM(64, return_sequences=True),
    layers.LSTM(32),
    layers.Dense(1, activation='sigmoid')
])
```

تجميع النموذج

```
model.compile(optimizer='adam',  
              loss='binary_crossentropy',  
              metrics=['accuracy'])
```

تدريب النموذج

```
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

تقييم النموذج

```
test_loss, test_acc = model.evaluate(x_test, y_test)  
print(f"Test accuracy: {test_acc:.4f}")
```

Using Keras in Regression

This example uses a neural network to predict numerical values using California Housing data.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.datasets import boston_housing

# تحميل بيانات السكن
(x_train, y_train), (x_test, y_test) = boston_housing.load_data()

# تطبيع البيانات
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

# بناء النموذج
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(x_train.shape[1],)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1)
])
```

تجميع النموذج

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

تدريب النموذج

```
model.fit(x_train, y_train, epochs=100, validation_data=(x_test, y_test))
```

تقييم النموذج

```
test_loss, test_mae = model.evaluate(x_test, y_test)
```

```
print(f"Test MAE: {test_mae:.4f}")
```

Converting the trained model into a usable model (save and load)

After training a model, we can save and load it later for use without the need to retrain.

```
# حفظ النموذج
model.save("my_model.keras")

# تحميل النموذج
new_model = keras.models.load_model("my_model.keras")

# اختبار النموذج المحمل
test_loss, test_acc = new_model.evaluate(x_test, y_test)
print(f"Loaded model accuracy: {test_acc:.4f}")
```



Task

Create a Model with Multiple Inputs & Outputs

Use Data Augmentation with Keras

Train a Model with a Learning Rate Scheduler